

R para Ciência de Dados (2ª edição)

Índice

Boas-vindas	3
I Transformar	5
1 Vetores lógicos	8

Boas-vindas

Este é o website para a tradução em Português da [2ª edição do livro “R for Data Science”](#). Este livro vai te ensinar como fazer ciência de dados com R: você irá aprender como importar os seus dados para o R, arrumá-los em uma estrutura mais útil, transformá-los e visualizá-los.

Sobre a tradução em Português

A tradução da 2ª edição do livro “**R for Data Science**” para o Português é um projeto colaborativo da comunidade de R no Brasil, que tem como objetivo tornar o R mais acessível na região.

A tradução foi feita por um grupo de pessoas voluntárias. Participaram das etapas de tradução e revisão do livro as seguintes pessoas (em ordem alfabética): Alberson Miranda, Ariana Cabral, Arthur C. Silva, Beatriz C. Colvara, Beatriz Milz, Bruno Miotto, Carolina Musso, Cesar A. Galvao, Debora Silva, Eric Scopinho, Fernanda Fiel Peres, Halian Vilela, Jeanne Franco, José Carlos Barbosa, Luana Antunes Alexandre, Lígia Louzada, Magno Tairone, Marcus Ramalho e Maximilian Canez Fernandes. Agradecemos pelo trabalho!

Agradecemos a todas as pessoas que ajudaram revisando as traduções e fazendo sugestões de melhoria. Também agradecemos os autores do livro original, [por permitirem a tradução e a disponibilização do conteúdo de forma gratuita](#).

A coordenação geral da tradução foi feita por Beatriz Milz. Riva Quiroga, a coordenadora da [tradução da versão em espanhol](#), atuou como consultora do projeto.

Este projeto não envolve apenas a tradução do texto, mas também dos conjuntos de dados usados ao longo dele. Para isso, foi criado o pacote [dados](#) (disponível no CRAN), que contém as versões traduzidas desses conjuntos. O pacote foi baseado em seu “pacote irmão” em espanhol: [datos](#).

Sobre a versão original

Neste livro, você vai encontrar um conjunto de habilidades para a ciência de dados. Assim como estudantes de química aprendem a limpar tubos de ensaio e montar um laboratório, você vai aprender a limpar dados e fazer gráficos – além de muitas outras coisas. Essas são

as habilidades que fazem a ciência de dados acontecer, e aqui você vai encontrar as melhores práticas para fazer cada uma dessas coisas com o R. Você vai aprender a usar a gramática dos gráficos, programação letrada (*literate programming*) e pesquisa reprodutível para economizar tempo. Você também vai aprender a gerenciar recursos cognitivos para facilitar descobertas ao lidar com, visualizar e explorar dados.

Este site é e sempre será gratuito, licenciado sob a Licença [CC BY-NC-ND 3.0](#). Se você gostaria de ter uma cópia física do livro, você pode comprar a versão em Inglês na [Amazon](#). Se você aprecia ler o livro gratuitamente e gostaria de retribuir, por favor faça uma doação para a [Kākāpō Recovery](#): o *kākāpō* (que aparece na capa do livro) é uma espécie de papagaio nativa da Nova Zelândia que está criticamente ameaçada de extinção; restam apenas 248 espécimes.

Você pode encontrar respostas sugeridas para exercícios no livro em <https://mine-cetinkaya-rundel.github.io/r4ds-solutions>.

Se você fala outra língua, pode se interessar pelas traduções gratuitas da primeira edição do livro:

- [Espanhol](#)
- [Italiano](#)
- [Turco](#)

Por favor, note que o R4DS usa um [Código de Conduta para Contribuidores](#). Ao contribuir com este livro, você concorda em seguir estes termos.

Agradecimentos

A versão original do livro (em Inglês) é hospedada pela <https://www.netlify.com> como parte de seu apoio ao software e às comunidades *open source*.

Parte I

Transformar

A segunda parte do livro abordou, com profundidade, a visualização de dados. Nesta parte do livro, você aprenderá sobre os tipos mais importantes de variáveis que você encontrará dentro de um *data frame* e conhecerá as ferramentas que pode usar para trabalhar com essas variáveis.

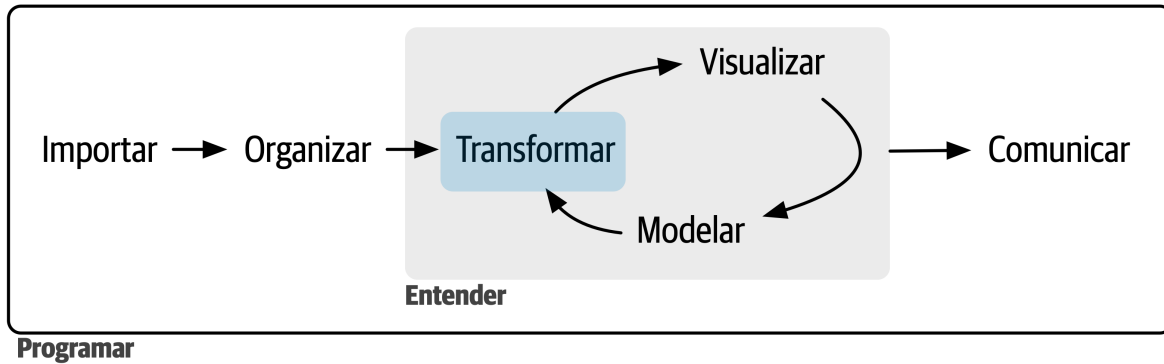


Figura 1: As opções de transformação de dados dependem bastante do tipo de dados envolvidos, que é o foco desta parte do livro.

Você pode ler esses capítulos de acordo com a sua necessidade; eles foram escritos para serem independentes, permitindo que sejam lidos sem uma ordem específica.

- O Capítulo 1 descreve os vetores lógicos. Estes são os tipos mais simples de vetores, porém são extremamente poderosos. Neste capítulo, você aprenderá a criar esses vetores lógicos por meio de comparações numéricas, a combiná-los utilizando álgebra booleana, a empregá-los para fazer sumarizações e a aplicá-los em transformações condicionais.
- O ?@sec-numbers explora ferramentas para vetores numéricos, a coluna vertebral da ciência de dados. Neste capítulo, você aprenderá mais sobre contagem e uma variedade de funções importantes de transformação e sumarização.
- O ?@sec-strings fornece as ferramentas necessárias para trabalhar com cadeias de caracteres (*strings*): você conseguirá extrair partes de uma *string*, separar uma *string* em vários caracteres e concatená-los para gerar uma *string* novamente. Este capítulo dá ênfase principalmente ao pacote stringr, mas também aborda algumas funções do tidyrr voltadas para a extração de dados de *strings*.
- O ?@sec-regular-expressions introduz as expressões regulares, uma ferramenta poderosa para manipular cadeias de caracteres (*strings*). Este capítulo fará com que você saia da fase de pensar que um gato caminhou sobre o seu teclado e passe a ler e escrever com facilidade padrões complexos de *strings*.
- O ?@sec-factors introduz os fatores: o tipo de dado que o R utiliza para armazenar dados categóricos. Você utiliza um fator quando uma variável possui um conjunto fixo

de valores possíveis ou quando você deseja atribuir uma ordenação não alfabética a uma *string*.

- O **?@sec-dates-and-times** fornece as principais ferramentas para trabalhar com datas e horários. Infelizmente, quanto mais você aprende sobre datas e horários, parece que o assunto se tornar mais complicado. No entanto, com a ajuda do pacote *lubridate*, você aprenderá a superar os desafios mais frequentes.
- O **?@sec-missing-values** discute, em detalhes, os valores faltantes (*missing values*). Ao longo do livro, abordamos esse tema algumas vezes de forma isolada, mas agora é hora de uma análise mais abrangente que ajude você a distinguir valores ausentes implícitos de valores ausentes explícitos e a compreender como e por que você pode convertê-los.
- O **?@sec-joins** encerra esta parte do livro sobre transformação, fornecendo ferramentas para unir dois (ou mais) *data frames*. Entender sobre uniões (*joins*) fará com que você assimile o conceito de chaves (*keys*) de banco de dados e pense sobre como identificar cada linha em um conjunto de dados.

1 Vetores lógicos

1.1 Introdução

Neste capítulo, você aprenderá ferramentas para trabalhar com vetores lógicos. Vetores lógicos são a forma mais simples de vetores, pois cada elemento pode ter apenas um dos três possíveis valores: `TRUE`, `FALSE` e `NA`. É relativamente raro encontrar vetores lógicos em seus dados brutos, mas você os criará e manipulará no decorrer de quase todas as análises.

Começaremos discutindo a forma mais comum de criar vetores lógicos: por meio de comparações numéricas. Em seguida, você aprenderá como usar a álgebra booleana para combinar diferentes vetores lógicos, bem como algumas sumarizações úteis. Terminaremos com `if_else()` e `case_when()`, duas funções úteis para fazer alterações condicionais alimentadas por vetores lógicos.

1.1.1 Pré-requisitos

A maioria das funções que você aprenderá neste capítulo são fornecidas pelo R base, então não precisamos do tidyverse, mas ainda assim iremos carregá-lo para que possamos usar `mutate()`, `filter()`, e demais funções para trabalhar com *data frames*. Também continuaremos a extrair exemplos do conjunto de dados `dados::voos`.

```
library(tidyverse)
library(dados)
```

No entanto, à medida que começamos a estudar novas ferramentas, nem sempre haverá um exemplo real perfeito. Então, começaremos a criar alguns dados fictícios com `c()`:

```
x <- c(1, 2, 3, 5, 7, 11, 13)
x * 2
#> [1]  2  4  6 10 14 22 26
```

Isso facilita a explicação de funções individuais, ao custo de dificultar a visualização de como elas podem se aplicar aos seus problemas de dados. Apenas lembre-se de que qualquer transformação que fizemos em um vetor simples, você poderá fazer em uma variável dentro de *data frame* com `mutate()` e demais funções amigas.

```
df <- tibble(x)
df |>
  mutate(y = x * 2)
#> # A tibble: 7 x 2
#>       x     y
#>   <dbl> <dbl>
#> 1     1     2
#> 2     2     4
#> 3     3     6
#> 4     5    10
#> 5     7    14
#> 6    11    22
#> # i 1 more row
```

1.2 Comparações

Uma forma muito comum de criar um vetor lógico é através de uma comparação numérica usando `<`, `<=`, `>`, `>=`, `!=` e `==`. Até agora, criamos principalmente variáveis lógicas temporariamente dentro do `filter()` — elas são calculadas, usadas e depois jogadas fora. Por exemplo, o filtro abaixo encontra todos os voos que saíram durante o dia e chegaram aproximadamente no horário:

```
voos |>
  filter(horario_saida > 600 & horario_saida < 2000 & abs(atraso_chegada) < 20)
#> # A tibble: 172,286 x 19
#>   ano  mes  dia horario_saida saida_programada atraso_saida
#>   <int> <int> <int>         <int>         <int>         <dbl>
#> 1  2013     1     1           601             600           1
#> 2  2013     1     1           602             610          -8
#> 3  2013     1     1           602             605          -3
#> 4  2013     1     1           606             610          -4
#> 5  2013     1     1           606             610          -4
#> 6  2013     1     1           607             607           0
#> # i 172,280 more rows
#> # i 13 more variables: horario_chegada <int>, chegada_prevista <int>, ...
```

É útil saber que isto é apenas um atalho e você pode explicitamente criar a variável lógica intrínseca com `mutate()`:

```
voos |>
  mutate(
    diurno = horario_saida > 600 & horario_saida < 2000,
    aprox_no_horario = abs(atraso_chegada) < 20,
    .keep = "used"
  )
#> # A tibble: 336,776 x 4
#>   horario_saida atraso_chegada diurno aprox_no_horario
#>       <int>         <dbl> <lgl>   <lgl>
#> 1         517             11 FALSE    TRUE
#> 2         533             20 FALSE    FALSE
#> 3         542             33 FALSE    FALSE
#> 4         544            -18 FALSE    TRUE
#> 5         554            -25 FALSE    FALSE
#> 6         554             12 FALSE    TRUE
#> # i 336,770 more rows
```

Isto é particularmente útil para lógicas mais complicadas, pois nomeando os passos intermediários, se torna mais fácil tanto ler o código quanto verificar se cada passo está sendo calculado corretamente.

Com isto, o filtro inicial é equivalente a:

```
voos |>
  mutate(
    diurno = horario_saida > 600 & horario_saida < 2000,
    aprox_no_horario = abs(atraso_chegada) < 20,
  ) |>
  filter(diurno & aprox_no_horario)
```

1.2.1 Comparações com ponto-flutuante (*floating point*)

Cuidado ao usar `==` com números. Por exemplo, parece que este vetor contém os números 1 e 2:

```
x <- c(1 / 49 * 49, sqrt(2) ^ 2)
x
#> [1] 1 2
```

Mas se você fizer um teste de igualdade, você terá `FALSE`:

```
x == c(1, 2)
#> [1] FALSE FALSE
```

O que está acontecendo aqui? Os computadores armazenam números com um número fixo de casas decimais, então não há como representar exatamente $1/49$ ou `sqrt(2)` e os cálculos subsequentes serão ligeiramente imprecisos. Podemos ver os valores exatos chamando `print()` com o argumento `digits`¹:

```
print(x, digits = 16)
#> [1] 0.9999999999999999 2.0000000000000004
```

Você pode ver porque o R arredonda estes números por padrão: eles são realmente muito próximos ao que você espera.

Agora que você viu porque `==` falhou, o que você pode fazer a respeito? Uma opção é usar `dplyr::near()` que ignora pequenas diferenças:

```
near(x, c(1, 2))
#> [1] TRUE TRUE
```

1.2.2 Valores faltantes (*missing values*)

Os valores faltantes (*missing values*) representam o desconhecido, portanto são “contagiosos”: quase qualquer operação que envolva um valor desconhecido também será desconhecida:

```
NA > 5
#> [1] NA
10 == NA
#> [1] NA
```

O resultado mais confuso é este:

```
NA == NA
#> [1] NA
```

É mais fácil entender porque isto é verdadeiro se nós artificialmente fornecermos um pouco mais de contexto:

¹R normalmente chama *print* para você (por exemplo, `x` é um atalho para `print(x)`), mas chamando-o explicitamente é útil se você quer fornecer outros argumentos.

```
# Não sabemos a idade de Maria
idade_maria <- NA

# Não sabemos a idade de João
idade_joao <- NA

# João e Maria tem a mesma idade?
idade_maria == idade_joao
#> [1] NA
# Não sabemos!
```

Portanto, se você quiser encontrar todos os voos onde `horario_saida` está faltando, o código a seguir não funciona, pois `horario_saida == NA` retorna `NA` para cada linha e `filter()` ignora automaticamente valores faltantes:

```
voos |>
  filter(horario_saida == NA)
#> # A tibble: 0 x 19
#> #   i 19 variables: ano <int>, mes <int>, dia <int>, horario_saida <int>,
#> #     saida_programada <int>, atraso_saida <dbl>, horario_chegada <int>, ...
```

Ao invés disso, você precisará de uma nova ferramenta: `is.na()`.

1.2.3 `is.na()`

`is.na(x)` funciona com qualquer tipo de vetor e retorna `TRUE` para valores faltantes e `FALSE` para qualquer outra coisa:

```
is.na(c(TRUE, NA, FALSE))
#> [1] FALSE  TRUE FALSE
is.na(c(1, NA, 3))
#> [1] FALSE  TRUE FALSE
is.na(c("a", NA, "b"))
#> [1] FALSE  TRUE FALSE
```

Podemos usar `is.na()` para encontrar todos os registros com `horario_saida` faltando:

```
voos |>
  filter(is.na(horario_saida))
#> # A tibble: 8,255 x 19
```

```
#>      ano  mes  dia horario_saida saida_programada atraso_saida
#>   <int> <int> <int>         <int>         <int>         <dbl>
#> 1  2013    1    1           NA           1630           NA
#> 2  2013    1    1           NA           1935           NA
#> 3  2013    1    1           NA           1500           NA
#> 4  2013    1    1           NA            600           NA
#> 5  2013    1    2           NA           1540           NA
#> 6  2013    1    2           NA           1620           NA
#> # i 8,249 more rows
#> # i 13 more variables: horario_chegada <int>, chegada_prevista <int>, ...
```

`is.na()` também pode ser útil em `arrange()`. `arrange()` geralmente coloca todos os valores faltantes no final, mas você pode sobrescrever este padrão ordenando primeiro com `is.na()`:

```
voos |>
  filter(mes == 1, dia == 1) |>
  arrange(horario_saida)
#> # A tibble: 842 x 19
#>      ano  mes  dia horario_saida saida_programada atraso_saida
#>   <int> <int> <int>         <int>         <int>         <dbl>
#> 1  2013    1    1           517           515           2
#> 2  2013    1    1           533           529           4
#> 3  2013    1    1           542           540           2
#> 4  2013    1    1           544           545          -1
#> 5  2013    1    1           554           600          -6
#> 6  2013    1    1           554           558          -4
#> # i 836 more rows
#> # i 13 more variables: horario_chegada <int>, chegada_prevista <int>, ...

voos |>
  filter(mes == 1, dia == 1) |>
  arrange(desc(is.na(horario_saida)), horario_saida)
#> # A tibble: 842 x 19
#>      ano  mes  dia horario_saida saida_programada atraso_saida
#>   <int> <int> <int>         <int>         <int>         <dbl>
#> 1  2013    1    1           NA           1630           NA
#> 2  2013    1    1           NA           1935           NA
#> 3  2013    1    1           NA           1500           NA
#> 4  2013    1    1           NA            600           NA
#> 5  2013    1    1           517           515           2
#> 6  2013    1    1           533           529           4
```

```
#> # i 836 more rows
#> # i 13 more variables: horario_chegada <int>, chegada_prevista <int>, ...
```

Voltaremos a abordar mais detalhes sobre valores faltantes (*missing values*) no `?@sec-missing-values`.

1.2.4 Exercício

1. Como `dplyr::near()` funciona? Digite `near` para ver o código fonte. `sqrt(2)^2` é próximo (near) a 2?
2. Use `mutate()`, `is.na()` e `count()` juntos para descrever como os valores faltantes (*missing values*) em `horario_saida`, `saida_programada` e `atraso_saida` estão relacionados.

1.3 Álgebra booleana

Quando você tem múltiplos vetores lógicos, você pode combiná-los usando álgebra booleana. No R, `&` é “e”, `|` é “ou”, `!` é “não” e `xor()` é ou exclusivo². Por exemplo, `df |> filter(!is.na(x))` encontra todas as linhas onde `x` está faltando e `df |> filter(x < -10 | x > 0)` encontra todas as linhas onde `x` é menor que -10 ou maior que 0. A Figura 1.1 mostra o conjunto completo de operações booleanas e como elas funcionam.

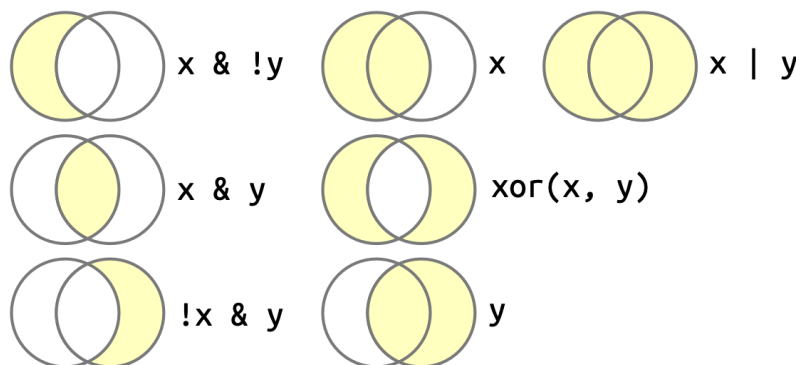


Figura 1.1: O conjunto completo de operações booleanas. `x` é o círculo da esquerda, `y` é o círculo da direita e a região preenchida mostra qual parte cada operação seleciona.

Além de `&` e `|`, o R também possui `&&` e `||`. Não os use em funções `dplyr`! Eles são chamados de operadores de curto-circuito e retornam sempre apenas um único `TRUE` ou `FALSE`. Eles são importantes na programação, não em ciência de dados.

²Isto é, `xor(x, y)` é verdadeiro se `x` for verdadeiro, ou `y` for verdadeiro, mas nunca ambos. Isto é como usamos “ou” em Português. “Ambos” geralmente não é uma resposta aceitável quando respondemos a pergunta “Você gostaria de sorvete ou bolo?”.

1.3.1 Valores faltantes (*missing values*)

As regras para valores faltantes na álgebra booleana são um pouco complicadas de explicar porque parecem inconsistentes à primeira vista:

```
df <- tibble(x = c(TRUE, FALSE, NA))

df |>
  mutate(
    e = x & NA,
    ou = x | NA
  )
#> # A tibble: 3 x 3
#>   x     e     ou
#>   <lgl> <lgl> <lgl>
#> 1 TRUE  NA   TRUE
#> 2 FALSE FALSE NA
#> 3 NA    NA   NA
```

Para entender o que está acontecendo aqui, pense em $NA \mid TRUE$ (NA ou TRUE). Um valor faltante (*missing value*) em um vetor lógico significa que o valor poderia ser TRUE ou FALSE. $TRUE \mid TRUE$ e $FALSE \mid TRUE$ são ambos TRUE pois ao menos um dos termos é TRUE. $NA \mid TRUE$ também deve ser TRUE pois NA pode ser TRUE ou FALSE. Entretanto, $NA \mid FALSE$ é NA pois não sabemos se NA é TRUE ou FALSE. O mesmo se aplica com $NA \& FALSE$.

1.3.2 Ordem das operações

Observe que a ordem das operações não funcionam como em Português. Veja o seguinte código que encontra todos os voos que saíram em novembro ou dezembro:

```
voos |>
  filter(mes == 11 | mes == 12)
```

Você pode estar tentado a escrevê-lo da forma que falaria em Português: “Encontre todos os voos que saíram em Novembro e Dezembro.”:

```
voos |>
  filter(mes == 11 | 12)
#> # A tibble: 336,776 x 19
#>   ano  mes  dia horario_saida saida_programada atraso_saida
#>   <int> <int> <int>         <int>             <int>           <dbl>
```

```
#> 1  2013      1      1          517          515          2
#> 2  2013      1      1          533          529          4
#> 3  2013      1      1          542          540          2
#> 4  2013      1      1          544          545         -1
#> 5  2013      1      1          554          600         -6
#> 6  2013      1      1          554          558         -4
#> # i 336,770 more rows
#> # i 13 more variables: horario_chegada <int>, chegada_prevista <int>, ...
```

Este código não gera erro, mas também não parece ter funcionado. O que acontece aqui? Neste caso, o R calcula `mes == 11` criando um vetor lógico, o qual chamaremos de `nov`. Então calcula `nov | 12`. Quando usamos um número com um operador lógico, ele converte tudo exceto 0 para `TRUE`, o que é equivalente a `nov | TRUE` o que é sempre `TRUE`, então cada linha será selecionada:

```
voos |>
  mutate(
    nov = mes == 11,
    final = nov | 12,
    .keep = "used"
  )
#> # A tibble: 336,776 x 3
#>   mes nov  final
#>   <int> <lgl> <lgl>
#> 1     1 FALSE TRUE
#> 2     1 FALSE TRUE
#> 3     1 FALSE TRUE
#> 4     1 FALSE TRUE
#> 5     1 FALSE TRUE
#> 6     1 FALSE TRUE
#> # i 336,770 more rows
```

1.3.3 %in%

Um jeito fácil de evitar o problema de colocar `==` e `is` na ordem correta é usar `%in%`. `x %in% y` retorna um vetor lógico de mesmo tamanho que `x` que é `TRUE` sempre que um valor em `x` está em qualquer lugar em `y`.

```
1:12 %in% c(1, 5, 11)
#> [1]  TRUE FALSE FALSE FALSE  TRUE FALSE FALSE FALSE FALSE FALSE  TRUE FALSE
```

```
letters[1:10] %in% c("a", "e", "i", "o", "u")
#> [1] TRUE FALSE FALSE FALSE TRUE FALSE FALSE FALSE TRUE FALSE
```

Portanto, para encontrar todos os voos de novembro e dezembro, poderíamos escrever:

```
voos |>
  filter(mes %in% c(11, 12))
```

Note que `%in%` possui diferentes regras do NA para `==`, uma vez que `NA %in% NA` é `TRUE`.

```
c(1, 2, NA) == NA
#> [1] NA NA NA
c(1, 2, NA) %in% NA
#> [1] FALSE FALSE TRUE
```

Isto pode ser um atalho útil:

```
voos |>
  filter(horario_saida %in% c(NA, 0800))
#> # A tibble: 8,803 x 19
#>   ano   mes   dia horario_saida saida_programada atraso_saida
#>   <int> <int> <int>         <int>         <int>         <dbl>
#> 1  2013     1     1           800           800           0
#> 2  2013     1     1           800           810          -10
#> 3  2013     1     1           NA          1630           NA
#> 4  2013     1     1           NA          1935           NA
#> 5  2013     1     1           NA          1500           NA
#> 6  2013     1     1           NA           600           NA
#> # i 8,797 more rows
#> # i 13 more variables: horario_chegada <int>, chegada_prevista <int>, ...
```

1.3.4 Exercícios

1. Encontre todos os voos em que `atraso_chegada` está faltando (*missing*), mas `atraso_saida` não. Encontre todos os voos em que nem `horario_chegada` ou `chegada_prevista` estão faltando, mas `atraso_chegada` está.
2. Quantos voos possuem `horario_saida` faltando? Quais outras variáveis possuem valores faltantes nestes registros? O que podem representar estas linhas?
3. Assumindo que um valor faltante em `horario_saida` implica em um voo cancelado, olhe o número de voos cancelados por dia. Há algum padrão? Há alguma relação entre a proporção de voos cancelados e a média de atraso dos voos não cancelados?

1.4 Sumarização

As seções a seguir descrevem técnicas úteis para sumarização de vetores lógicos. Assim como funções que trabalham especificamente com vetores lógicos, você também pode usar funções que trabalham com vetores numéricos.

1.4.1 Sumarizações lógicas

Existem dois sumarizadores lógicos: `any()` e `all()`. `any(x)` é o equivalente ao `|`; retornará `TRUE` se houver algum `TRUE` em `x`. `all(x)` é o equivalente ao `&`; retornará `TRUE` somente se todos os valores de `x` forem `TRUE`. Assim como todas as funções de sumarização, elas retornarão `NA` se houver qualquer valor faltante (*missing value*) presente, e como de costume, você pode não considerá-los usando `na.rm = TRUE`.

Por exemplo, poderíamos usar `all()` e `any()` para descobrir se todos os voos foram atrasados na partida por no máximo uma hora ou se algum voo atrasou na chegada por cinco horas ou mais. E usando `group_by()` podemos fazer isso para cada dia:

```
voos |>
  group_by(ano, mes, dia) |>
  summarize(
    todos_atrasados = all(atraso_saida <= 60, na.rm = TRUE),
    algum_atraso_longo = any(atraso_chegada >= 300, na.rm = TRUE),
    .groups = "drop"
  )
#> # A tibble: 365 x 5
#>   ano   mes   dia todos_atrasados algum_atraso_longo
#>   <int> <int> <int> <lgl>          <lgl>
#> 1  2013     1     1 FALSE          TRUE
#> 2  2013     1     2 FALSE          TRUE
#> 3  2013     1     3 FALSE          FALSE
#> 4  2013     1     4 FALSE          FALSE
#> 5  2013     1     5 FALSE          TRUE
#> 6  2013     1     6 FALSE          FALSE
#> # i 359 more rows
```

Na maioria dos casos, entretanto, `any()` e `all()` são um tanto grosseiros, e seria bom poder obter mais detalhes sobre os valores `TRUE` ou `FALSE`. Isso nos leva aos sumarizadores numéricos.

1.4.2 Sumarizações numéricas de vetores lógicos

Quando você usa um vetor lógico em um contexto numérico, `TRUE` se torna 1 e `FALSE` se torna 0. Isto torna `sum()` e `mean()` muito úteis com vetores lógicos, pois `sum(x)` retorna o número de `TRUE`s e `mean(x)` retorna a proporção de `TRUE`s (pois `mean()` é `sum()` dividido por `length()`).

Isto, por exemplo, nos permite ver a proporção de voos que tiveram atraso na saída em até uma hora e o número de voos que chegaram atrasados em cinco horas ou mais:

```
voos |>
  group_by(ano, mes, dia) |>
  summarize(
    todos_atrasados = mean(atraso_saida <= 60, na.rm = TRUE),
    algum_atraso_longo = sum(atraso_chegada >= 300, na.rm = TRUE),
    .groups = "drop"
  )
#> # A tibble: 365 x 5
#>   ano   mes   dia todos_atrasados algum_atraso_longo
#>   <int> <int> <int>         <dbl>         <int>
#> 1  2013     1     1           0.939             3
#> 2  2013     1     2           0.914             3
#> 3  2013     1     3           0.941             0
#> 4  2013     1     4           0.953             0
#> 5  2013     1     5           0.964             1
#> 6  2013     1     6           0.959             0
#> # i 359 more rows
```

1.4.3 Subconjunto (*subsetting*) lógico

Existe ainda um último uso de vetores lógicos em sumarizações: você pode usar um vetor lógico para filtrar uma única variável em um subconjunto (*subset*) de interesse. Isto faz uso do operador do R base `[` (se pronuncia *subset*), o qual você aprenderá mais na [?@sec-subset-many](#).

Imagine que gostaríamos de encontrar a média de atraso somente de voos que realmente estiveram atrasados. Uma forma de fazê-lo seria primeiro filtrar os voos e depois calcular a média de atraso:

```
voos |>
  filter(atraso_chegada > 0) |>
  group_by(ano, mes, dia) |>
  summarize(
```

```

    atraso_medio = mean(atraso_chegada),
    n = n(),
    .groups = "drop"
  )
#> # A tibble: 365 x 5
#>   ano   mes   dia atraso_medio     n
#>   <int> <int> <int>       <dbl> <int>
#> 1  2013     1     1         32.5   461
#> 2  2013     1     2         32.0   535
#> 3  2013     1     3         27.7   460
#> 4  2013     1     4         28.3   297
#> 5  2013     1     5         22.6   238
#> 6  2013     1     6         24.4   381
#> # i 359 more rows

```

Isto funciona, mas e se você quisesse calcular o atraso médio também para voos que chegaram antecipadamente? Precisariamos fazer um passo extra de filtro e então combinar os dois *data frames*³. Ao invés disso, você poderia usar `[` para executar um filtro em linha (*inline*): `atraso_chegada[atraso_chegada > 0]` irá retornar apenas os atrasos positivos na chegada.

Isso leva a:

```

voos |>
  group_by(ano, mes, dia) |>
  summarize(
    atrasado = mean(atraso_chegada[atraso_chegada > 0], na.rm = TRUE),
    antecipado = mean(atraso_chegada[atraso_chegada < 0], na.rm = TRUE),
    n = n(),
    .groups = "drop"
  )
#> # A tibble: 365 x 6
#>   ano   mes   dia atrasado antecipado     n
#>   <int> <int> <int>       <dbl>       <dbl> <int>
#> 1  2013     1     1         32.5        -12.5   842
#> 2  2013     1     2         32.0        -14.3   943
#> 3  2013     1     3         27.7        -18.2   914
#> 4  2013     1     4         28.3        -17.0   915
#> 5  2013     1     5         22.6        -14.0   720
#> 6  2013     1     6         24.4        -13.6   832
#> # i 359 more rows

```

³Iremos cobrir isto na ?@sec-joins.

Observe também a diferença no tamanho do grupo: no primeiro segmento `n()` tem o número de voos em atraso por dia, no segundo, `n()` retorna o número total de voos.

1.4.4 Exercícios

1. O que `sum(is.na(x))` retorna? E `mean(is.na(x))`?
2. O que `prod()` retorna quando aplicada a um vetor lógico? Qual função de sumarização lógica é equivalente? O que `min()` retorna quando aplicada a um vetor lógico? Qual função de sumarização lógica é equivalente? Leia a documentação e efetue alguns experimentos.

1.5 Transformações condicionais

Uma das características mais poderosas de vetores lógico é seu uso em transformações condicionais, i.e. fazer alguma coisa em uma condição `x`, e alguma outra coisa diferente para a `y`. Existem duas funções importantes para isso: `if_else()` e `case_when()`.

1.5.1 `if_else()`

Se você quiser usar um valor para quando a condição é `TRUE` e outro valor para quando a condição é `FALSE`, você pode usar `dplyr::if_else()`⁴. Você sempre usará os primeiros três argumentos de `if_else()`. O primeiro, `condition`, é um vetor lógico, o segundo, `true`, gera uma saída quando a condição é verdadeira, e o terceiro, `false`, gera uma saída quando a condição é falsa.

Vamos começar com um simples exemplo rotulando um vetor numérico como “+vo” (positivo) ou “-vo” (negativo):

```
x <- c(-3:3, NA)
if_else(x > 0, "+vo", "-vo")
#> [1] "-vo" "-vo" "-vo" "-vo" "+vo" "+vo" "+vo" NA
```

Há um quarto argumento opcional, `missing` o qual pode ser usado se a entrada for um valor `NA`:

⁴A função `if_else()` do `dplyr` é muito similar a `ifelse()` do R base. Existem duas vantagens principais do `if_else()` sobre `ifelse()`: você pode escolher o que acontece com valor faltantes (*missing values*) e `if_else()` te retorna erros com mais sentido se sua variável possuir tipos incompatíveis.

```
if_else(x > 0, "+vo", "-vo", "???)
#> [1] "-vo" "-vo" "-vo" "-vo" "+vo" "+vo" "+vo" "???"
```

Você também usar vetores para os argumentos `true` e `false`. Por exemplo, isto nos permite criar uma implementação mínima de `abs()`:

```
if_else(x < 0, -x, x)
#> [1] 3 2 1 0 1 2 3 NA
```

Até agora, todos os argumentos usaram os mesmos vetores, mas você pode misturar e combinar variáveis. Por exemplo, você pode implementar uma versão simples de `coalesce()` desta forma:

```
x1 <- c(NA, 1, 2, NA)
y1 <- c(3, NA, 4, 6)
if_else(is.na(x1), y1, x1)
#> [1] 3 1 2 6
```

Você pode notar uma pequena infelicidade no nosso exemplo de rótulo acima: zero não é nem positivo nem negativo. Nós poderíamos resolver isso adicionando um novo `if_else()`:

```
if_else(x == 0, "0", if_else(x < 0, "-vo", "+vo"), "???)
#> [1] "-vo" "-vo" "-vo" "0" "+vo" "+vo" "+vo" "???"
```

Isto já é um pouco difícil de ler, mas você pode imaginar como se tornaria mais difícil se você tiver ainda mais condições. Ao invés disso, você deve mudar para o `dplyr::case_when()`.

1.5.2 case_when()

O `case_when()` do pacote `dplyr` é inspirado na declaração `CASE` do SQL e oferece uma forma flexível de efetuar diferentes cálculos para diferentes condições. Infelizmente, ele tem uma sintaxe que não se parece com nada que você usará no pacote `tidyverse`. Ele recebe pares que se parecem com `condição ~ saída`. `condição` deve ser um vetor lógico; quando for `TRUE`, `saída` será usada.

Isto significa que poderíamos recriar nos `if_else()` aninhado desta forma:

```
x <- c(-3:3, NA)
case_when(
  x == 0 ~ "0",
  x < 0 ~ "-vo",
  x > 0 ~ "+vo",
  is.na(x) ~ "???"
)
#> [1] "-vo" "-vo" "-vo" "0" "+vo" "+vo" "+vo" "???"
```

O código é maior, mas também é mais explícito.

Para explicar com `case_when()` funciona, vamos explorar alguns casos mais simples. Se nenhum dos casos corresponder, a saída será NA:

```
case_when(
  x < 0 ~ "-vo",
  x > 0 ~ "+vo"
)
#> [1] "-vo" "-vo" "-vo" NA "+vo" "+vo" "+vo" NA
```

Use o argumento `.default` se você quiser gerar um “padrão”/pegar todos os valores:

```
case_when(
  x < 0 ~ "-vo",
  x > 0 ~ "+vo",
  .default = "???"
)
#> [1] "-vo" "-vo" "-vo" "???" "+vo" "+vo" "+vo" "???"
```

E observe que se várias condições corresponderem, apenas a primeira será usada:

```
case_when(
  x > 0 ~ "+vo",
  x > 2 ~ "grande"
)
#> [1] NA NA NA NA "+vo" "+vo" "+vo" NA
```

Assim como com `if_else()` você pode usar variáveis em ambos os lados do `~` e pode misturar e combinar variáveis conforme necessário para o seu problema. Por exemplo, poderíamos usar `case_when()` para fornecer alguns rótulos legíveis para o atraso de chegada:

```

voos |>
  mutate(
    status = case_when(
      is.na(atraso_chegada) ~ "cancelado",
      atraso_chegada < -30 ~ "muito antecipado",
      atraso_chegada < -15 ~ "antecipado",
      abs(atraso_chegada) <= 15 ~ "no horario",
      atraso_chegada < 60 ~ "atrasado",
      atraso_chegada < Inf ~ "muito atrasado",
    ),
    .keep = "used"
  )
#> # A tibble: 336,776 x 2
#>   atraso_chegada status
#>       <dbl> <chr>
#> 1          11 no horario
#> 2          20 atrasado
#> 3          33 atrasado
#> 4         -18 antecipado
#> 5         -25 antecipado
#> 6          12 no horario
#> # i 336,770 more rows

```

Tenha cuidado ao escrever esse tipo de instrução `case_when()` complexa; minhas duas primeiras tentativas usaram uma mistura de `<` e `>` e continuei criando acidentalmente condições sobrepostas.

1.5.3 Tipos compatíveis

Observe que `if_else()` e `case_when()` requerem tipos **compatíveis** na saída. Se eles não forem compatíveis, você verá erros como este:

```

if_else(TRUE, "a", 1)
#> Error in `if_else()`:
#> ! Can't combine `true` <character> and `false` <double>.

case_when(
  x < -1 ~ TRUE,
  x > 0 ~ now()
)

```

```
#> Error in `case_when()`:  
#> ! Can't combine `..1 (right)` <logical> and `..2 (right)` <datetime<local>>.
```

No geral, relativamente poucos tipos são compatíveis, porque a conversão automática de um tipo de vetor em outro é uma fonte comum de erros. Aqui estão os casos mais importantes que são compatíveis:

- Vetores numéricos e lógicos são compatíveis, conforme discutimos na Seção 1.4.2.
- Strings e fatores (**?@sec-factors**) são compatíveis, porque você pode pensar em um fator como uma string com um conjunto restrito de valores.
- Data e data-hora, que discutiremos no **?@sec-dates-and-times**, são compatíveis porque você pode pensar em uma data (*date*) como um caso especial de data-hora (*datetime*).
- **NA**, que é tecnicamente um vetor lógico, é compatível com tudo porque todo vetor tem alguma forma de representar um valor faltante (*missing value*).

Não esperamos que você memorize essas regras, mas elas devem se tornar naturais com o tempo, porque são aplicadas de forma consistente em todo o tidyverse.

1.5.4 Exercícios

1. Um número é par se for divisível por dois, o que, no R, você pode descobrir com `x %% 2 == 0`. Use este fato e `if_else()` para determinar se cada número entre 0 e 20 é par ou ímpar.
2. Dado um vetor de dias como `x <- c("Segunda-feira", "Sábado", "Quarta-feira")`, use uma instrução `ifelse()` para rotulá-los como fins de semana ou dias de semana.
3. Use `ifelse()` para calcular o valor absoluto de um vetor numérico chamado `x`.
4. Escreva uma declaração `case_when()` que usa as colunas `mes` e `dia` de `voos` para rotular um conjunto de feriados importante dos Brasil (por exemplo: 7 de Setembro, Tiradentes, Carnaval e Natal⁵). Primeiro crie uma coluna lógica `TRUE` ou `FALSE`, e então uma coluna texto (*character*) que tenha o nome do feriado ou `NA`.

1.6 Resumo

A definição de vetor lógico é simples pois seu valor é `TRUE`, `FALSE` ou `NA`. Mas os vetores lógicos oferecem grande poder. Neste capítulo, você aprendeu a criar vetores lógicos com `>`, `<`, `<=`, `>=`, `==`, `!=` e `is.na()`, como combiná-los com `!`, `&` e `|`, e como sumariá-los com `any()`, `all()`,

⁵**Nota de tradução:** Na versão original em inglês, os feriados são dos Estados Unidos.

`sum()` e `mean()`. Você também aprendeu as funções poderosas `if_else()` e `case_when()` que permitem você retornar valores dependendo do valor de um vetor lógico.

Veremos vetores lógicos cada vez mais nos próximos capítulos. Por exemplo, no `?@sec-strings` você vai aprender sobre `str_detect(x, pattern)` que retorna um vetor lógico que será `TRUE` para elementos de `x` que correspondem ao `pattern` (padrão), e no `?@sec-dates-and-times` você irá criar vetores lógicos a partir de comparações de datas e horários. Por ora, iremos para o próximo mais importante tipo de vetores: vetores numéricos.